



Credit Card Fraud Detection

Final Model Comparison Report

Machine Learning Project • Meeting Presentation

Models compared: Logistic Regression • Decision Tree • Random Forest • KNN

Dataset: 1,000,000 transactions



Contents

Use the links below to jump directly to a section. The footer buttons also let you move through the report.

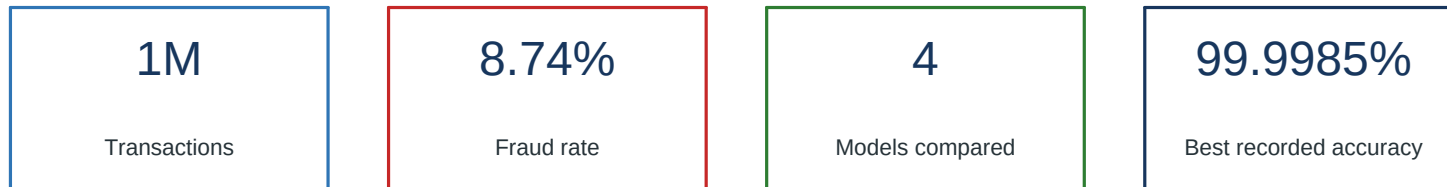
- [1. Project Overview](#)
- [2. Dataset & Data Preparation](#)
- [3. Class Imbalance](#)
- [4. Train-Test Split & Sampling](#)
- [5. Algorithms Used](#)
- [6. Logistic Regression](#)
- [7. Decision Tree](#)
- [8. Random Forest](#)
- [9. K-Nearest Neighbors](#)
- [10. Model Performance Comparison](#)
- [11. Confusion Matrix Comparison](#)
- [12. Best Model Selection](#)
- [13. Why Other Models Were Not Selected](#)
- [14. Limitations & Improvements](#)

[15. Overall Conclusion](#)[16. Presentation Answer](#)

Tip for your meeting: explain the business problem first, then show why fraud recall and false negatives matter more than accuracy alone.

1. Project Overview

The goal of this project is to build a machine learning system that can identify whether a credit card transaction is genuine or fraudulent.



Business Problem

Credit card fraud can cause financial loss to customers and financial institutions. Because thousands of transactions can happen every minute, checking every transaction manually is not practical. Machine learning can help automatically flag transactions that look suspicious.

Project Goal

Compare four classification algorithms and select the strongest model based on accuracy, precision, recall, F1-score and confusion matrix results.

Models: **Logistic Regression, Decision Tree, Random Forest and KNN.**

2. Dataset & Data Preparation

The project uses a dataset containing 1,000,000 credit card transactions. The notebook's extended dataset contains 12 columns.

Feature	Meaning
distance_from_home	Distance between transaction location and customer's home
distance_from_last_transaction	Distance from the previous transaction
ratio_to_median_purchase_price	Purchase value compared with the median purchase price
repeat_retailer	Whether the transaction was made at a previously used retailer
used_chip	Whether a chip was used
used_pin_number	Whether a PIN was used
online_order	Whether the order was online
fraud	Target: 0 = genuine, 1 = fraud

Data quality: The supplied project summary reports no missing values and 0 duplicate records.

Time features: Hour, Minute, Second, Transaction_Time and Time_Period were added. The report notes that these time values were randomly generated, so they should not be treated as actual historical transaction times.

3. Class Imbalance

Fraud detection is an imbalanced classification problem because genuine transactions are much more common than fraudulent transactions.

912,597	87,403	8.74%	91.26%
Genuine	Fraud	Fraud share	Genuine share

Imagine 100 transactions: around 91 are genuine and only around 9 are fraudulent.

Why imbalance matters

A model could predict most transactions as genuine and still get a high accuracy score. But in a fraud detection system, failing to detect an actual fraud is costly.

False Negative: the transaction is actually fraudulent, but the model predicts it as genuine.

For this project, fraud recall and the number of false negatives are especially important.

4. Train-Test Split & Sampling

Train-Test Split

The dataset was split into 80% training data and 20% testing data using `random_state=42` and `stratify=y`.

800,000	200,000	182,519	17,481
Training rows	Testing rows	Genuine in test	Fraud in test

Handling Class Imbalance

Random Under-Sampling: reduces the majority class to make the classes more balanced.

Random Over-Sampling: increases the minority class by duplicating minority examples. The notebook used `RandomOverSampler(random_state=42)`.

The oversampled training data reached **1,460,156 rows**. Only the training data was resampled; the test set remained unchanged.

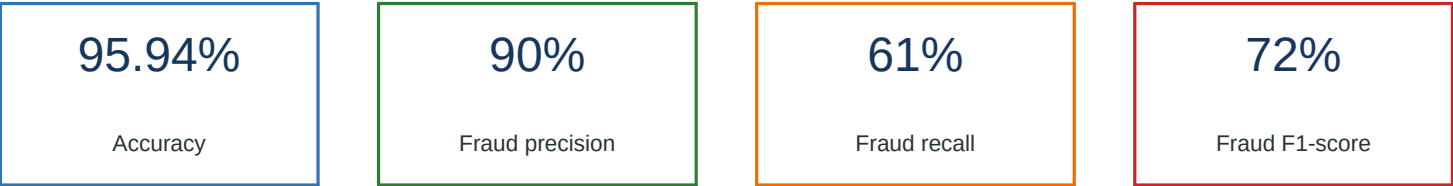
5. Algorithms Used

Algorithm	Simple idea	Role in this project
Logistic Regression	Calculates a probability for fraud vs genuine.	Baseline classification model.
Decision Tree	Makes a sequence of if-else decisions.	Strong, easy-to-understand classifier.
Random Forest	Combines predictions from many decision trees.	More robust tree-based model.
KNN	Looks at nearby/similar transactions.	Similarity-based classifier.

Think of the models as four different ways of answering the same question: **“Is this transaction likely to be fraud?”**

6. Logistic Regression

Logistic Regression predicts the probability that a transaction belongs to class 0 (genuine) or class 1 (fraud). The project standardized the features before training it.



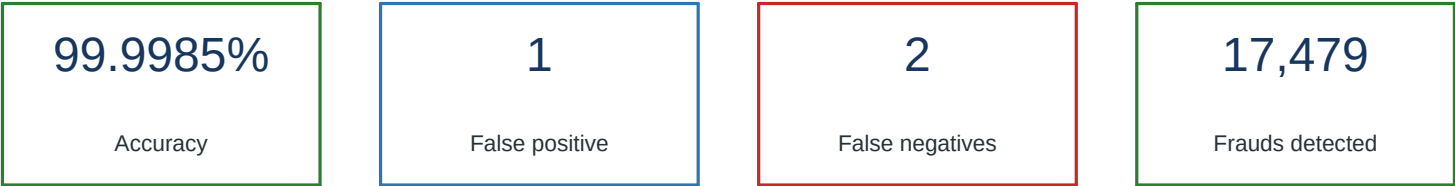
Confusion Matrix

	Predicted Genuine	Predicted Fraud
Actual Genuine	181,296	1,223
Actual Fraud	6,897	10,584

Main weakness: it missed **6,897 fraudulent transactions**. That is a major concern for a fraud detection application.

7. Decision Tree

A Decision Tree works like a sequence of if-else questions. It repeatedly splits transactions using feature conditions until it reaches a prediction.



Confusion Matrix

	Predicted Genuine	Predicted Fraud
Actual Genuine	182,518	1
Actual Fraud	2	17,479

On this test set, the Decision Tree missed only 2 fraudulent transactions and produced only 1 false positive.

8. Random Forest

Random Forest combines many Decision Trees. The project used **100 trees** (n_estimators=100). Each tree contributes to the final prediction.

99.9975%

Accuracy

0

False positives

5

False negatives

17,476

Frauds detected

Confusion Matrix

	Predicted Genuine	Predicted Fraud
Actual Genuine	182,519	0
Actual Fraud	5	17,476

Random Forest was also extremely strong. It made **zero false-positive errors**, but it missed 5 fraud cases—3 more than the Decision Tree.

9. K-Nearest Neighbors (KNN)

KNN predicts a new transaction by looking at the most similar nearby transactions. The notebook used `n_neighbors=5` and standardized the features.

99.551%

Accuracy

96%

Fraud precision

99%

Fraud recall

97%

Fraud F1-score

Confusion Matrix

	Predicted Genuine	Predicted Fraud
Actual Genuine	181,730	789
Actual Fraud	109	17,372

KNN detected most fraud cases, but it incorrectly flagged **789 genuine transactions** as fraud.

10. Model Performance Comparison

The following comparison uses the results recorded in the supplied project report.

Model	Accuracy	Fraud Precision	Fraud Recall	Fraud F1
Logistic Regression	95.94%	90%	61%	72%
Decision Tree	99.9985%	≈100%*	≈100%*	≈100%*
Random Forest	99.9975%	≈100%*	≈100%*	≈100%*
KNN	99.551%	96%	99%	97%



*The notebook's classification report rounds the tree-model class metrics to 1.00; the confusion matrices show the small number of underlying errors.

11. Confusion Matrix Comparison

For fraud detection, false negatives are especially important because they represent fraud that the system failed to catch.

Model	True Negative	False Positive	False Negative	True Positive
Logistic Regression	181,296	1,223	6,897	10,584
Decision Tree	182,518	1	2	17,479
Random Forest	182,519	0	5	17,476
KNN	181,730	789	109	17,372

False negatives: Logistic Regression = 6,897 • KNN = 109 • Random Forest = 5 • Decision Tree = 2

Based on the supplied results, Decision Tree has the lowest number of missed fraud transactions.

12. Best Model Selection

■ Based on the recorded test results, the selected model is the **Decision Tree**.

99.9985%

Accuracy

1

False positive

2

False negatives

17,479

True positives

Why this model was selected

- It achieved the highest recorded accuracy: 99.9985%.
- It produced only 1 false positive.
- It produced only 2 false negatives.
- It correctly identified 17,479 of the 17,481 fraudulent transactions in the test set.
- On this particular test split, it performed slightly better than Random Forest.

Important: this is the best model **for the results recorded in this project**. A production decision should use stronger validation before deployment.

13. Why Other Models Were Not Selected

Logistic Regression

It is simple and useful as a baseline, but its fraud recall was only 61% and it missed 6,897 fraud cases.

KNN

It achieved 99% fraud recall, but it generated 789 false positives. That means many genuine transactions could be incorrectly flagged.

Random Forest

It performed extremely well with 99.9975% accuracy and 0 false positives, but it missed 5 fraud cases versus only 2 for the Decision Tree.

Practical note: In a real production setting, Decision Tree vs Random Forest should not be decided from one test split alone. Cross-validation, hyperparameter tuning and newer unseen data should also be checked.

14. Limitations & Improvements

Limitations

- The added transaction time values were randomly generated, so time-based findings should not be treated as historical facts.
- Decision Tree and Random Forest produced almost perfect scores; this unusually high performance should be investigated.
- The model uses the available transaction features and does not include richer real-world signals such as device, IP, merchant and customer-history information.
- Accuracy alone is not enough for an imbalanced fraud problem.

Recommended next steps

- Run cross-validation.
- Tune Decision Tree and Random Forest hyperparameters.
- Evaluate on newer, unseen transaction data.
- Re-run Logistic Regression evaluation after the resampling step so the post-balancing result is cleanly measured.
- Monitor precision, recall, F1-score and false negatives—not just accuracy.

15. Overall Conclusion

This project demonstrated how machine learning can be used to identify fraudulent credit card transactions. The dataset contained 1 million transactions, with 87,403 fraudulent records (approximately 8.74%).

Four algorithms were compared: Logistic Regression, Decision Tree, Random Forest and KNN. Logistic Regression provided a reasonable baseline but missed many fraud cases. KNN detected most fraud cases but produced more false positives. Random Forest was extremely strong, while the Decision Tree achieved the best recorded result on this test set.

Final selection: Decision Tree — 99.9985% accuracy, 1 false positive and 2 false negatives.

Before real-world financial deployment, the model should undergo further validation, tuning and testing on new unseen data.



16. Presentation Answer

If your mentor asks: “**What is the conclusion of your project?**”

“In my credit card fraud detection project, I analyzed 1 million transactions and found that around 8.74% were fraudulent. Since the dataset was imbalanced, I applied sampling techniques to improve the model’s ability to learn the minority fraud class. I compared Logistic Regression, Decision Tree, Random Forest and KNN. Among these models, Decision Tree gave the best result on my test dataset, achieving 99.9985% accuracy with only 2 false negatives and 1 false positive. Therefore, I selected Decision Tree as the best model for this project. However, I would perform further validation and testing before using it in a real-world fraud detection system.”

Meeting Tip

Keep your explanation simple: **Problem** → **Data** → **Imbalance** → **Models** → **Comparison** → **Best Model** → **Limitations** → **Conclusion**.

This report is designed for presentation: use the Contents links to jump to a section, and use the footer buttons to navigate.